PCPOP : A Julia library for polynomial optimization with partially commutative variables

Moisés Bermejo Morán[2] and Abhishek Mishra[1]

[1]Laboratoire d'Information Quantique, Université libre de Bruxelles, Belgium
[2]Bilkent University, Türkiye

September 25, 2025

PCPOP implements mathematical structures and new algorithms to represent and manipulate elements of free algebra, partially-commutative algebra, graph product algebra and space of trace polynomials. The main objective is to leverage the equality constraints to reduce the complexity of the SDP relaxations for polynomial optimization problems. Given the set of equality constriants G with subset S of them being special binomial constraints such as partial commutations, projectors, unitaries, dichotomic, orthogonality etc., PCPOP finds normal forms directly and efficiently and then for the remaining equality constraints $G \setminus S$, it uses the theory of Gröbner basis in the quotient space modulo S to reduce the polynomials and hence reduce the complexity of the SDP relaxations. For trace polynomial optimization, it finds normal forms with resepct to cyclic equivalence, partial commutations and other special binomial constraints.

1 Introduction

PCPOP is a Julia library that builds and solves SDP relaxations for polynomial optimization with partially-commutative variables, where some variables commute and some not, and hence generalizing the non-commutative and fully-commutative polynomial optimization. It supports both the eigen-value and tracial versions.

Polynomial optimization has become a central tool in many diverse fields, but it is a NP-hard problem. Lassere first developed a numerical algorithm to solve the fully-commutative polynomial optimization problems. This was later extended to Non-Commutative variables yielding the NPA hierarchy [**<empty citation>**]. Given the natural non-commutativity of operators in quantum physics, the NPA hierarchy finds numerous applications in quantum foundations, device-independent quantum cryptography, many-body physics, and others. It has also been extended to trace polynomials, with many prominent applications, such as in quantum networks and quantum cryptography in prepare and measure scenarios.

Given $p(\underline{x}), q_i(\underline{x})$, polynomials in $\underline{x}$=$(x_1, x_2 ... x_n)$ variables, the fully-commutative polynomial optimization reads as,

$$\begin{aligned} p^* \;=\; & \inf_{\underline{x}} p(\underline{x}), \\ & s.t\; q_i(\underline{x}) \geq 0. \end{aligned} \tag{1}$$

In non-commutative variables $\mathbf{X}=(\mathbf{x}_1, \mathbf{x}_2 ... \mathbf{x}_n)$ and polynomials $p(X), q_i(X), r_i(X)$ in free algebra, the eigen-value version of non-commutative polynomial optimization reads as,

$$\begin{aligned} p^* = \inf_{\underline{A}\in\mathcal{B}(\mathcal{H})^n, \psi\in\mathcal{H},\mathcal{H}} & \langle \psi, p(\underline{A})\psi \rangle, \\ s.t\ & q_i(\underline{A}) \geq 0, \\ s.t\ & \langle \psi, r_i(\underline{A})\psi \rangle \geq 0, \\ s.t\ & \langle \psi, \psi \rangle = 1. \end{aligned} \tag{2}$$

In non-commutative variables $\mathbf{X}=(\mathbf{x}_1, \mathbf{x}_2 ... \mathbf{x}_n)$ and polynomials $p(X), q_i(X), r_j(X)$, in free algebra, the tracial version of non-commutative polynomial optimization reads as,

$$\begin{aligned} p^* = & inf\ tr(p(X)), \\ s.t\ & q_i(X) \geq 0. \\ & tr(r_j(X)) \geq 0 \end{aligned} \tag{3}$$

Prtially commutative polynomial optimization is a polynomial optimization in the partially-commuative algebra[see [<**empty citation**>] for details]. Informally speaking, given set of variables X and a graph $\mathcal{G} = (X, E)$, whose vertices are letters that are connected if they commute, we denote by $X^{\mathcal{G}}$ the partially-commutative variables and the partially commutative polynomial optimization reads as,

$$\begin{aligned} p^* = \inf_{\underline{A}\in\mathcal{B}(\mathcal{H})^n, \psi\in\mathcal{H},\mathcal{H}} & \langle \psi, \ulcorner p \urcorner (\underline{A})\psi \rangle, \\ s.t\ & \ulcorner q_i \urcorner (\underline{A}) \geq 0, \\ s.t\ & \langle \psi, \ulcorner r_i \urcorner (\underline{A})\psi \rangle \geq 0, \\ s.t\ & \langle \psi, \psi \rangle = 1. \end{aligned} \tag{4}$$

where $\ulcorner p \urcorner$ means the polynomial with partially-commutative variables. The tracial version can be defined similarly as,

$$\begin{aligned} p^* = & inf\ tr(\ulcorner p \urcorner (X)), \\ s.t\ & \ulcorner q_i \urcorner (X) \geq 0. \\ & tr(\ulcorner r_j \urcorner (X)) \geq 0 \end{aligned} \tag{5}$$

The algorithms for solving polynomial optimization problems, whether commutative or non-commutative, and their tracial or eigenvalue versions, share the same core idea of building a sequence of SDP relaxations of increasing size, whose optimal values converge to the global optimum under certain assumptions. The SDP relaxations are often indexed by a level parameter, with higher levels yielding better approximations of the global optimum but at the cost of increased computational complexity.

1.1 Motivation for PCPOP

Given how intractable it can be to solve the SDPs for higher levels or bigger problem sizes, i.e., with bigger PSD matrices and a large number of PSD and scalar constraints, several solutions to reduce the problem complexity by harnessing the structure of the input problem have been proposed. For e.g in, they use the sparsity of the input problem to reduce the SDP size at each levels of hierarchy, in [<**empty citation**>] , they propose how to use symmetry in the input problem to reduce the problem size and in [<**empty citation**>] they propose efficient solution for special type of correlations called synchronous correlations.

The main motivation for PCPOP comes from polynomial optimization problems with equality constraints that appear ubiquitously in physics, especially in the field of quantum information.

We found mathematical tools that abstracts these ubiquitous scenarios and PCPOP implements the associated mathematical structures and algorithms to efficiently build and solve the SDP relaxations. Below we discuss three classes of problems that motivated us to build PCPOP

For the first class, consider a quantum experiment, where some operators act jointly on quantum systems(say an unitary describing interaction of two quantum systems) and some operators act locally (say measurement on a quantum system). So operators that share a quantum system don't commute with each other but operators that don't share a quantum system , that is, they act on space-like separated quantum systems commute with each other. The operators could also satisfy other binomial constraints such as being projectors, unitary, dichotomic, pauli constraints or some pairs of operators being orthogonal. These operators can be abstracted as partially-commutative variables. And the polynomial optimization consisting of such operators can be best solved as a partially-commutative polynomial optimization problem[see **??** for examples and how they are implemented].

For the second class, consider a polynomial optimization problem which consists of two types of equality constraints. First being, special commutation rules such that the variables can be partitioned into different subsets where all the variables in a subset share the same commutation relations. The other type of constraints consist of polynomials that act on variables belonging to the same subset. We discuss methods in [**<empty citation>**] to leverage such type of equality constraints as efficiently as possible. The abstract mathematical structure used for this is the Graph product algebra which extends the partially-commutative algebra. See **??** for examples and how they are implemented.

The third class of problems consists of trace polynomial optimization problems as in **??** but with partially commutative variables and the equality constraints requiring the variables to be projectors, unitaries, or dichotomic. This class of problems appear naturally when considering prepare and measure scenrios. We discuss methods in [**<empty citation>**] to build efficient SDP relaxations for such class of problems. See **??** for examples and how they are implemented.

1.2 Scope of PCPOP

Here we discuss all the classes of polynomial optimization problems that can be solved using PCPOP.

- Non-commutative polynomial optimization problems, as mentioned in 2 and illustrated in section **??**. PCPOP uses AbstractAlgebra.jl package to reduce the polynomials using non-commutative Gröbner basis.
- Partially-commutative polynomial optimization problems, as mentioned in 4 and illustrated in section **??**. Here the special binomial constraints such as projectors, unitaries, dichotomic, orthogonality etc. are taken care implicitly while building the monomials as clique normal forms.
- Graph product polynomial optimization problems.
- Tracial polynomial optimization problems with partially commuting variables, as mentioned in 5 and illustrated in **??**. Here the special binomial constraints such as projectors, unitaries, dichotomic, orthogonality etc. are taken care implicitly while building the monomials as cyclic graph normal forms.

1.3 How PCPOP compares to other related libraries

Ncpol2sdpa is a python-based library for solving non-commutative polynomial optimization problems. It is popular in the quantum information community as it was the first library to implement the NPA hierarchy with easy interface to fully express a non-commutative polynomial

optimization problem. Ncpol2sdpa has some shortcomings as mentioned below, that PCPOP tries to overcome.

- It is python-based, and hence slower than Julia-based libraries.
- It implicitly assumes normalization and it is hard to tweak that.
- Trace polynomial optimization[**??**] is not implemented.
- Support for leveraging symmetries in the optimization is very limited.
- General equality constraints are implemented using localizing matrices which is not efficient and binomial constraints are handled using replacement rules which isn't effective [see **??** for an example].

QuantumNPA is another library for implementing NPA hierarchy. It is Julia-based and has easier interface to express common quantum information scenarios. It also has some shortcomings that PCPOP tries to overcome.

- Support for leveraging symmetries in the optimization problem is very limited.
- Trace polynomial optimization[**??**] is not implemented.
- Hard to impose general equality constraints.
- It can handle partially-commutative varibles that satisfy common binomial constraints such as being projector, unitary or dichotomic but it can be hard to express those and not efficient for larger problems.

Another popular Julia-based library is SumOfSquares which is excellent for fully-commutative polynomial optimization problems and has support for leveraging sparsity and symmetry in the input problem. It has some support for non-commutative polynomial optimization problems with shortcomings that PCPOP tries to overcome.

- Hard to impose general equality constraints.
- Lack of support for quantum information specific interface or simplifications.
- Trace polynomial optimization[**??**] is not implemented.
- It lacks support for leveraging symmetries in the optimization problem for the non-commutative case.

2 Technical background

PCPOP implements the monomials in partially-commutative monoid through the *clique normal forms* and *graph normal forms* as discussed in [**<empty citation>**]. Here we briefly introduce and illustrate these normal forms.

Given a set of variables $X = \{x_1, x_2, \ldots, x_n\}$, and a graph $\mathcal{G} = (X, E)$ with the vertices as the variables and the edge set capturing the independence relations, i.e $(x_i, x_j) \in E$ if and only if x_i and x_j don't commute. This graph is called the independence graph. The dependence graph is the complement of the independence graph. To find the *clique-normal form* $w^{\mathcal{C}}$ for a word w, we project the word onto the maximal cliques of the dependence graph. This has been illustrated though the example below.

Example 1. *Say $X = \{a, b, c\}$ and $[a, b] = [b, c] = 0$ are the only commutations. The dependence graph is as shown below.*

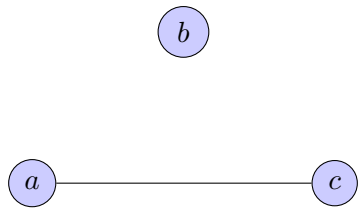

Figure 1: Dependence graph

The maximal cliques are $\{b\}, \{a, c\}$. Now consider the words $w_1 = cab$ and $w_2 = bca$. The clique normal forms are $w_1^{\mathcal{C}} = w_2^{\mathcal{C}} = (ca, b)$.

To compute the graph normal form of a word w, we label the different occurances of each letter in the word w from left to right. For eg. for a word $w = abacb$, we label it as $a_1b_1a_2c_1b_2$. Then we build the directed graph w^G whose vertices are the labeled letters and there is a directed edge $x \to y$ if the corresponding letters of the labels x, y don't commute and if there is not path between them. It is illustrated as follows.

Example 2. *Say $X = \{a, b, c\}$ and $[a, b] = [b, c] = 0$ are the only commutations. Consider the word $w = abacb$. We label it as $a_1b_1a_2c_1b_2$. The graph w^G is as shown below.*

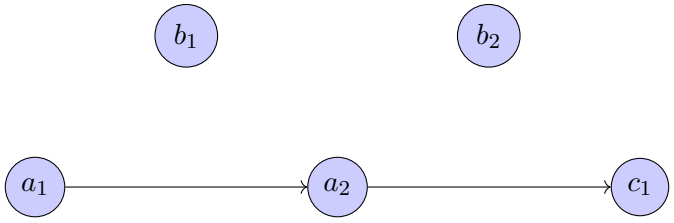

Figure 2: Graph w^G

PCPOP implements data structures to represent polynomials in graph product algebras and some new algorithms to find normal forms for certian type of graph product algebras and class of equality constriants[see [**<empty citation>**] for details]. A graph product algebra is the span of monomials belonging to a graph product monoid. Informally speaking, a graph product monoid is a monoid that combines different monoids where some monoids commute and some not. Partially-commutative monoid can be seen as special instance of grpah product algebra by repalcing the monoids with letters.

PCPOP implements the elements of partially commutative monoid and graph product monoid using same data structure called `GraphProductWord{V}` where V takes the type for variables when it partially-commutative word and type for monoid elements when it is a graph product word. The data structure `GraphProductWord{V}` essentially consists of three fields, first being an array of array `clique_words` to hold the clique normal form and the other two are sets(`edge_l,edge_r`) that hold the letters or monoid elements that can be commuted to the edge(left, right edge). Having the fields `edge_l, edge_r` helps in efficiently implementing the multiplication of two graph product words. Below is an example, that illustrates how to find normal forms under partial commutations and projectors.

Example 3. *Say we have two quantum systems that belong to Alice and Bob respectively, Alice has two projective measurements $\{A_0, A_1\}$ and Bob has two projective measurements $\{B_0, B_1\}$ and there are two projectors $\{AB_0, AB_1\}$ that act on both Alice's and Bob's systems. We know*

that Alice's and Bob's measurements commute with each other but not with the joint projectors. The operators belonging to Alice forms the free monoid $\mathbb{M}_A$*, similarly, operators of Bob form* $\mathbb{M}_B$ *and the joint operators form* $\mathbb{M}_{AB}$*. We combine the monoids* $\{\mathbb{M}_A, \mathbb{M}_B, \mathbb{M}_{AB}\}$ *where* $\mathbb{M}_A, \mathbb{M}_B$ *commute to form the graph product monoid* $\mathbb{M}$*. Informally speaking, the maximal cliques are* $\{\mathbb{M}_A, \mathbb{M}_{AB}\}$ *and* $\{\mathbb{M}_B, \mathbb{M}_{AB}\}$*.*

We can rewrite the words $w_1 = AB_0B_0A_1A_0$ *and* $w_2 = A_1B_0AB_1$ *as*

$w_1 = (AB_0)_{\mathbb{M}_{AB}}(B_0)_{\mathbb{M}_B}(A_1A_0)_{\mathbb{M}_A}$ *and* $w_2 = (A_1)_{\mathbb{M}_A}(B_0)_{\mathbb{M}_B}(AB_1)_{\mathbb{M}_{AB}}$*, where we multiply adjacent letters that belong to the same monoid and represented the result as word in respective monoid.*

Now If we multiply them, we get $w_1w_2 = (AB_0)_{\mathbb{M}_{AB}}(B_0)_{\mathbb{M}_B}(A_1A_0A_1)_{\mathbb{M}_A}(AB_1)_{\mathbb{M}_{AB}}$*, as* B_0 *can commute through* A_1 *and* A_0*. In their clique normal forms, the multiplication algorithm is illustrated below.*

$$\underset{\{AB_0\}}{\overset{w_1}{\begin{bmatrix}(AB_0)(A_1A_0)\\ (AB_0)(B_0)\end{bmatrix}}} \underset{\{(B_0),\ (A_1A_0)\}}{} \times \underset{\{(A_1),(B_0)\}}{\overset{w_2}{\begin{bmatrix}(A_1)(AB_1)\\ (B_0)(AB_1)\end{bmatrix}}} \underset{\{(AB_1)\}}{}$$

$$=$$

$$\underset{\{AB_0\}}{\overset{w'_1}{\begin{bmatrix}(AB_0)(A_1A_0)\\ (AB_0)\end{bmatrix}}} \underset{\{(A_1A_0)\}}{} \times \underset{\{(B_0)\}\ \{(B_0)\}}{\overset{m}{\begin{bmatrix}\\ (B_0)\end{bmatrix}}} \times \underset{\{(A_1),(AB_1)\}\ \{(AB_1)\}}{\overset{w'_2}{\begin{bmatrix}(A_1)(AB_1)\\ (AB_1)\end{bmatrix}}}$$

$$=$$

$$\underset{w_1 * w_2}{\begin{bmatrix}(AB_0)(A_1A_0A_1)(AB_1)\\ (AB_0)(B_0)(AB_1)\end{bmatrix}}$$

Figure 3: Multiplication of graph product monomials (column vector view). Two words in their clique normal forms are multiplied. The clique normal forms are shown as column vectors(within square brackets) where each row has the projection of the word to the respective maximal clique. The words within simple brackets represent that they belong to a different monoids. The set below and left(right) to the clique normal forms represent the monoid elements that can be commuted to the left(right) edge. To multiply w_1, w_2 we check which monoid elements on the right edge of w_1 can be multiplied with the left edge elements of w_2. We represent the result of the multiplication in the middle as m. We update the clique normal forms and the left and right edge sets of w_1, w_2 accordingly to $w'_1w'_2$. We then multiply $w'_1 * m * w'_2$ and we continue iteratively.

3 Getting started with PCPOP

This secion is divided in four parts, starting with how to solve classic non-commutative polynomial optimization and then moving to partially-commutative polynomial optimization, then the graph product polynomial optimization and finally trace partailly-commutative polynomial optimization.

3.1 Non-commutative polynomial optimization

Consider the following problem with hermitian variables X_1, X_2 from [**<empty citation>**] and already solved using Ncpol2sdpa.

$$\begin{aligned} p^* = \inf_{\underline{X},\psi,\mathcal{H}} \; & \langle \psi, X_1X_2 + X_2X_1\psi \rangle, \\ s.t \quad & X_1^2 - X_1 = 0, \\ & -X_2^2 + X_2 + 1/2 \geq 0, \\ & \langle \psi, \psi \rangle = 1. \end{aligned} \tag{6}$$

The julia code to solve the above problem using PCPOP is given below.

```
# To define the non-commutative variables X1,X2 and its monoid M
@ncmonoid M X1 X2
#  To define the equality constraints
Projector(X1)
build(M)

# To define the objective function, inequality constraints and level of
relaxation
obj=X1*X2+X2*X1
op_ge=[X2-X2^2+0.5]
level=2

# To solve the non-commutative polynomial optimization problem
ov,model,var_dict,principal_matrix=npa(obj,level;op_ge=op_ge)

println("Optimal value is ",ov)
```

To solve a non-commutative polynomial optimization problem using PCPOP, we follow the follwoing steps

- First we need to define the non-commutative variables and an object that stores information about the corresponding monoid using the macro `@ncmonoid`. The first argument is always for the monoid object and the rest for the variables.
- Then we define the equality constraints using `add_relations!`. For constriants like *projector, Unitary, Unipotent* we can use built-in functions `Projector,Unitary,Unipotent` respectively.
- Finally, we call the function `npa` to solve the optimization problem. `npa` takes the objective polynomial and the level as compulsory arguments and also some optional arguments like arrays of operator inequalities(`op_ge`), operator equalities(`op_eq`), expectation value equalities(`tr_eq`) and inequalities(`tr_ge`). The function returns the optimal value, the SDP model, a dictionary that maps the monomial basis to coressponding JuMP variables and finally the principal matrix.

PCPOP provides nice interface to declare variables. We can create a container of variables using `X[n,m]` where X is the name for variable array, n is the number of hermitian variables and m is the number of non-hermitian variables. In that case X stores the hermitian variables and $X\_$ stores the non-hermitian variables.

```
julia> @ncmonoid M X[2,1] a_
julia> X
2-element Vector{cgb.Variable}:
X1
```

```
X_2
julia> X_
1-element Vector{cgb.Variable}:
X_1_
julia>X_[1]'
X_1__
julia> cgb.is_herm(a_)
false
julia> a_'
a__
```

We can add relations using the built-in function `add_relations!` which takes as input, an array of non-commutative polynomials. When the monoid is build using `build(M)`, the `AbstractAlgebra` package is used to compute the Gröbner basis for the ideal generated by the relations and then it is used to reduce the polynomials. It is illustrated below.

```
julia>@ncmonoid M a b
julia>cgb.add_relations!(M,[a^2 - a - 1, a * b - b * a])
julia>cgb.build(M)
julia>a*b*a*a
2.0b * a + 1.0b
```

3.2 Partially-commutative polynomial optimization

Consider the problem of certifying genuine multi-partite non-locality in a Bell scenario with n parties where each party performs two dichotomic measurements [AMR24]. Here we illustrate the case for two parties. This can be cast as a polynomial optimization problem with variables $X = \{A_1, A_2, B_1, B_2\}$ each being projectors and where A_i, B_j belong to Alice and Bob respectively. The problem reads as follows.

$$\begin{aligned} p^* = \inf_{\underline{X},\psi,\mathcal{H}} \; & \langle \psi, A_1 B_1 \psi \rangle, \\ s.t \quad & [A_i, B_j] = 0, \forall i,j \in \{1,2\} \\ s.t \quad & A_i^2 - A_i = B_j^2 - B_j = 0, \forall i,j \in \{1,2\} \\ s.t \quad & \langle \psi, A_2 B_1 \psi \rangle = \langle \psi, A_1 B_2 \psi \rangle = 0, \\ s.t \quad & \langle \psi, (1 - A_2)(1 - B_2)\psi \rangle = 0, \\ & \langle \psi, \psi \rangle = 1. \end{aligned} \tag{7}$$

This can be solved with PCPOP as follows.

```
@pcmonoid M A[2,0] B[2,0]
Projector.([A;B])
@comms A B
build(M)
obj=A[1]*B[1]
tr_eq= [[A[2] * B[1],0],[A[1] * B[2],0],[(1-A[2]) * (1-B[2]),0]]
level=2
ov,model,var_dict,principal_matrix=npa(obj,level;tr_eq=tr_eq;min=false)
println("Optimal value is ",ov)
```

THis returns optimal value as 0.0902 as in [AMR24]. The steps to solve a partially-commutative polynomial optimization problem are similar to that of non-commutative polynomial optimization with some differences.

- First we define the partially-commutative monoid using the macro `@pcmonoid`. The first argument is always for the monoid object and the rest for the variables.

- Then, we define which variables commute using the built-in macro `@comms`.
- Then we define some special operator equality constraints like *projector*, *unitary*, *dichotomic* and *orthogonal pairs* using built-in functions `Projector,Unitary,Unipotent` and macro `@ortho` respectively. We leave the general equality constraints to be fed to `npa` function as the optional argument `op_eq`.
- Finally, we call the function `npa` to solve the optimization problem. `npa` takes the objective polynomial and the level as compulsory arguments and also some optional arguments like arrays of operator inequalities(`op_ge`), operator equalities(`op_eq`), expectation value equalities(`tr_eq`) and inequalities(`tr_ge`). The function returns the optimal value, the SDP model, a dictionary that maps the monomial basis to coressponding JuMP variables and finally the principal matrix.

Remark 4. *Notice, that compared to the non-commutative case, where we we enforce all the equality constraint before building the monoid, here we only enforce the special equality constraints like projector, unitary, dichotomic and orthogonality before building the monoid. These special equality constraints are directly and efficiently handled using normal forms and not using Macaulay method which handles the general operator equality constraints.*

The user interface to define the variables and the monoid is same as in non-commutative case. The macro `@comms` provides multiple ways to define commutation relations. We illustrate all the options.

```
julia>@pcmonoid M a b # a and b are two hermitian variables
julia>@comms a b #  variable a commutes with variable b
julia>print(a*b,"\n",b*a)
(a)(b)
(a)(b)
julia> a*b==b*a
true
```

```
julia> @pcmonoid M A[2,0] B[2,0] # A1,A2 are hermitian variables for Alice and
     B1,B2 for Bob
julia> @comms A B # all variables of A commute with all variables of B
```

```
julia>@pcmonoid M a b c d e f # a,b,c,d,e,f are six hermitian variables
julia>@comms [a,b,c] # to enforce [a,b]=[b,c]=[a,c]=0
julia>a*b*c==c*a*b
true
julia>a*e*b == b*e*a
false
```

To enforce the orthogonality constraint $a * b = 0$, we can use the built-in macro `@ortho` as `@ortho a b`. The `@ortho` macro works similarly as `@comms` macro.

3.3 Graph product polynomial optimization

References

[AMR24] Ranendu Adhikary, Abhishek Mishra, and Ramij Rahaman. "Self-testing of genuine multipartite entangled states without network assistance". In: *Phys. Rev. A* 110 (1 July 2024), p. L010401. DOI: 10.1103/PhysRevA.110.L010401. URL: https://link.aps.org/doi/10.1103/PhysRevA.110.L010401.